

Spec — mini-moi Monitoring Stack (Part 2)

Created: 2026-06-18 — Claude.ai Status: Ready for `/working/` — build after WebSocket spec (Part 1) is stable on AWS Depends on: AWS Phase 2 (EC2 deployed), Part 1 (WebSocket analysis)

Context

mini-moi is a production system in daily use, being positioned as enterprise-applicable. It requires professional monitoring — not Telegram summaries, not manual log checks, but a proper observability stack that surfaces problems before they affect the user and provides the data needed to improve the system over time.

Three monitoring concerns:

- 1. **Error tracking** — when something breaks, know immediately with full context. What failed, why, how often, which user session.
- 2. **Application metrics** — how is the system performing? Analysis job success rates, response times, WebSocket connection counts, Curator loop durations.
- 3. **Infrastructure metrics** — is the EC2 healthy? CPU, memory, disk, RDS connections, network.

The stack:

Tool	Layer	Why
Sentry	Error tracking	Captures every exception automatically with full context. Free tier (5k errors/month). Zero configuration after SDK init.
Prometheus	Metrics collection	Scrapes <code>/metrics</code> from Flask apps every 15s. Open source, standard.
Grafana	Dashboards + alerting	Reads Prometheus and CloudWatch. Professional dashboards. Claude Code builds them.
CloudWatch	AWS infrastructure	Native AWS metrics — EC2, RDS, S3. No extra cost. Feeds into Grafana.

This is the same stack used in professional engineering teams. Claude Code handles all configuration. Robert reads the dashboards and acts on alerts.

Part 2A — Sentry (error tracking)

What it does

Sentry captures every unhandled exception across all three Flask apps with full context: stack trace, request data, user session, frequency, affected versions. When an analysis job fails, Sentry has the full picture within seconds — no log hunting required.

Installation

```
bash

pip install sentry-sdk[flask]
```

Add to `requirements.txt` for all three apps.

Integration — one init per app

In each Flask app's `app.py` or init file:

```
python

import sentry_sdk
from sentry_sdk.integrations.flask import FlaskIntegration
from sentry_sdk.integrations.sqlalchemy import SQLAlchemyIntegration

sentry_sdk.init(
    dsn=os.environ['SENTRY_DSN'],
    integrations=[
        FlaskIntegration(),
        SQLAlchemyIntegration(),
    ],
    # Capture performance traces on 10% of requests
    traces_sample_rate=0.1,
    # Tag all events with the app name
    environment=os.environ.get('FLASK_ENV', 'production'),
)
```

`SENTRY_DSN` is the project-specific URL from Sentry's dashboard. Add to `.env.example` and AWS Secrets Manager.

Three Sentry projects

One project per Flask app:

- `minimoi-portal`
- `minimoi-curator`
- `minimoi-mein-deutsch`

Separate projects means separate error streams — a Curator failure doesn't appear in the German domain's error list.

Custom context for analysis jobs

In the WebSocket analysis background thread, add Sentry context so failures include the job details:

```
python

import sentry_sdk

def _run_analysis(job_id, transcript, persona, scene, model,
                  socket_id, session_id):
    with sentry_sdk.push_scope() as scope:
        scope.set_tag('job_id', job_id)
        scope.set_tag('persona', persona)
        scope.set_tag('model', model)
        scope.set_context('analysis_job', {
            'job_id': job_id,
            'session_id': session_id,
            'transcript_length': len(transcript),
            'scene': scene
        })
        try:
            result = run_review(...)
            ...
        except Exception as e:
            sentry_sdk.capture_exception(e)
            raise
```

When a job fails, Sentry shows: which persona, which model, transcript length, full stack trace. Diagnosable in 30 seconds.

Alerting

In Sentry dashboard: configure alerts to send to email (or Slack if added later) when:

- A new error type appears for the first time
- An existing error exceeds 10 occurrences in 1 hour

- Any error in the `analysis` module

These are Sentry's built-in alert rules — no custom code needed.

Definition of Done — Part 2A

- Sentry SDK installed in all three Flask apps
 - `SENTRY_DSN` in `.env.example` and AWS Secrets Manager
 - Three Sentry projects created (one per app)
 - Analysis job context added to background thread
 - Test: trigger a deliberate exception → confirm Sentry captures it with full context within 30 seconds
 - Alert rule created: notify on new error types
-

Part 2B — Prometheus (metrics collection)

What it does

Prometheus scrapes a `/metrics` endpoint on each Flask app every 15 seconds and stores the time-series data. Grafana reads from Prometheus to draw graphs and set alert thresholds.

Your Flask apps don't know about Grafana. They just expose numbers at `/metrics`. Prometheus and Grafana handle everything else.

Installation

```
bash

pip install prometheus-flask-exporter
```

Integration — one init per app

```
python
```

```
from prometheus_flask_exporter import PrometheusMetrics

metrics = PrometheusMetrics(app)

# Default metrics added automatically:
# - flask_http_request_total (request counts by endpoint + status)
# - flask_http_request_duration_seconds (response time histograms)
# - flask_http_request_exceptions_total (exception counts)
```

This adds `/metrics` to the Flask app automatically. Prometheus scrapes it. Zero additional code for basic HTTP metrics.

Custom metrics — what to instrument

Beyond the default HTTP metrics, instrument the things that matter:

Mein Deutsch — analysis jobs:

```
python

from prometheus_client import Counter, Histogram, Gauge

analysis_jobs_total = Counter(
    'gesprache_analysis_jobs_total',
    'Total analysis jobs submitted',
    ['model', 'persona', 'status'] # labels
)

analysis_duration = Histogram(
    'gesprache_analysis_duration_seconds',
    'Analysis job duration in seconds',
    ['model'],
    buckets=[5, 10, 15, 20, 30, 60]
)

websocket_connections = Gauge(
    'gesprache_websocket_connections_active',
    'Currently active WebSocket connections'
)
```

Usage:

```
python
```

```
# In _run_analysis():
with analysis_duration.labels(model=model).time():
    result = run_review(...)

analysis_jobs_total.labels(
    model=model,
    persona=persona,
    status='success'
).inc()

# On WebSocket connect/disconnect:
websocket_connections.inc() # connect
websocket_connections.dec() # disconnect
```

Curator — scoring loop:

```
python

curator_loop_duration = Histogram(
    'curator_loop_duration_seconds',
    'Curator scoring loop duration'
)

curator_articles_scored = Counter(
    'curator_articles_scored_total',
    'Articles scored by the Curator loop',
    ['source', 'status']
)
```

Portal — CoS tasks:

```
python

cos_tasks_total = Counter(
    'cos_tasks_total',
    'CoS periodic tasks completed',
    ['task_type', 'status']
)
```

Prometheus configuration

`prometheus.yml` — tells Prometheus where to scrape:

```
yaml
```

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'minimoi-portal'
    static_configs:
      - targets: ['localhost:5001']

  - job_name: 'minimoi-curator'
    static_configs:
      - targets: ['localhost:8766']

  - job_name: 'minimoi-mein-deutsch'
    static_configs:
      - targets: ['localhost:8767']

# CloudWatch metrics via prometheus-cloudwatch-exporter
- job_name: 'cloudwatch'
  static_configs:
    - targets: ['localhost:9106']
```

Running Prometheus on EC2

Add to `docker-compose.prod.yml`:

```
yaml

prometheus:
  image: prom/prometheus:latest
  volumes:
    - ./prometheus.yml:/etc/prometheus/prometheus.yml
    - prometheus_data:/prometheus
  ports:
    - "9090:9090"
  restart: unless-stopped
```

Prometheus UI at `http://localhost:9090` (internal only — not exposed to the internet).

Definition of Done — Part 2B

- `prometheus-flask-exporter` installed in all three apps
- `/metrics` endpoint responds on all three apps

- Custom metrics instrumented: analysis jobs, duration, WebSocket connections, Curator loop, CoS tasks
 - Prometheus running in Docker on EC2
 - `prometheus.yml` scraping all three apps successfully
 - Verify: `curl localhost:9090/api/v1/targets` shows all three apps as "up"
-

Part 2C — Grafana (dashboards + alerting)

What it does

Grafana reads from Prometheus (and CloudWatch) and draws dashboards. Claude Code builds the dashboards. Robert opens Grafana to read them.

Running Grafana on EC2

Add to `docker-compose.prod.yml`:

```
yml
grafana:
  image: grafana/grafana:latest
  ports:
    - "3000:3000"
  volumes:
    - grafana_data:/var/lib/grafana
    - ./grafana/provisioning:/etc/grafana/provisioning
  environment:
    - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_PASSWORD}
    - GF_SERVER_ROOT_URL=https://grafana.minimoi.ai
  restart: unless-stopped
```

Add `GRAFANA_PASSWORD` to `.env.example` and AWS Secrets Manager.

Subdomain

Add `grafana.minimoi.ai` to Cloudflare DNS → EC2. Nginx config:

```
nginx
```



```
server {  
    server_name grafana.minimoi.ai;  
    location / {  
        proxy_pass http://localhost:3000;  
        proxy_set_header Host $host;  
    }  
    listen 443 ssl;  
    # ... SSL config same as app.minimoi.ai  
}
```

Grafana accessible at <https://grafana.minimoi.ai>. Login with admin credentials from Secrets Manager.

Dashboards (Claude Code builds these)

Dashboard 1 — mini-moi Overview The first dashboard Robert sees on login:

- System health: all three apps up/down (green/red)
- Analysis jobs today: total, success rate, avg duration
- WebSocket connections: current active
- Curator last run: time ago, articles scored, errors
- Error count by app (24h) — links to Sentry

Dashboard 2 — Gespräche Analysis

- Analysis jobs per hour (time series)
- Success vs failure rate (pie chart)
- Duration percentiles: p50, p95, p99 (by model)
- Error types from Sentry webhook
- Active WebSocket connections over time

Dashboard 3 — Infrastructure Powered by CloudWatch via the CloudWatch exporter:

- EC2 CPU utilization (t3.small)
- EC2 memory usage
- RDS connections and CPU
- S3 request rates
- Network in/out

Dashboard 4 — Curator

- Scoring loop duration over time

- Articles scored per run
- Source breakdown
- Error rate by source

Alerts in Grafana

Alert rules fire when thresholds are crossed. Notification channel: email initially (configured in Grafana → Alerting → Contact Points). Add Slack or PagerDuty later if needed.

Alert rules:

```
Analysis success rate < 90% over 1 hour  
→ "Analysis failure rate elevated"
```

```
Analysis p95 duration > 45 seconds  
→ "Analysis taking too long – check model provider"
```

```
EC2 CPU > 80% for 10 minutes  
→ "EC2 CPU high – check active jobs"
```

```
RDS connections > 80% of max  
→ "Database connection pool near limit"
```

```
Any app returning 0 successful requests for 5 minutes  
→ "App may be down"
```

```
WebSocket connections = 0 for 30 minutes during active hours  
→ "WebSocket may have dropped"
```

CloudWatch integration

Install the CloudWatch Prometheus exporter as a container:

```
yaml
```

```
cloudwatch-exporter:
  image: prom/cloudwatch-exporter:latest
  volumes:
    - ./cloudwatch-exporter-config.yml:/config/config.yml
  ports:
    - "9106:9106"
  environment:
    - AWS_REGION=us-east-1
  restart: unless-stopped
```

This pulls EC2, RDS, and S3 metrics from CloudWatch into Prometheus so they appear in Grafana alongside application metrics.

Definition of Done — Part 2C

- Grafana running on EC2, accessible at <https://grafana.minimoi.ai>
 - Login secured with password from Secrets Manager
 - Four dashboards built and rendering correctly
 - All Prometheus data sources connected
 - CloudWatch exporter running, infrastructure metrics visible
 - Six alert rules configured
 - Test alert: artificially trigger a threshold → confirm alert fires
 - Verify: Robert can log into Grafana and read all four dashboards without configuration knowledge
-

Deployment notes for AWS

All monitoring containers run on the same EC2 instance as the Flask apps. At mini-moi scale this is fine — Prometheus and Grafana are lightweight. If the instance gets crowded, move them to a dedicated [t3.micro](#) monitoring instance later.

Port summary (internal only, not exposed):

- Prometheus: 9090
- Grafana: 3000 (exposed via Nginx at grafana.minimoi.ai)
- CloudWatch exporter: 9106

Data persistence:

- [prometheus_data](#) Docker volume — metrics history

- `grafana_data` Docker volume — dashboard definitions, alert rules
 - Both backed up to S3 weekly (add to backup script)
-

Operations runbook additions

Add to `docs/OPERATIONS.md` (to be written):

Checking system health:

1. Open `https://grafana.minimoi.ai`
2. Dashboard 1 (Overview) is the first view — all apps green, no active alerts
3. If an alert has fired, Grafana shows it on the dashboard with a link to the relevant panel

Checking a specific failure:

1. Grafana Dashboard 2 → find the failed job time window
2. Open Sentry → filter by the same time window → find the exception
3. Sentry shows full stack trace, job context, transcript length, model used

Adding a new metric:

1. Add `Counter`, `Histogram`, or `Gauge` to the relevant Flask app
 2. Instrument the code path
 3. Prometheus picks it up automatically on next scrape (15s)
 4. Add a panel to the relevant Grafana dashboard
-

Full monitoring commit sequence

Three commits, in order:

1. ``Add Sentry error tracking to all three Flask apps.``
 2. ``Add Prometheus metrics – HTTP, analysis jobs, Curator, CoS.``
 3. ``Add Grafana dashboards and alerts – four dashboards, six alert rules.``
-

Definition of Done — Full monitoring stack

- Sentry capturing exceptions in all three apps with custom context

- `/metrics` on all three apps, Prometheus scraping successfully
 - Grafana at `https://grafana.minimoi.ai` with four dashboards
 - CloudWatch infrastructure metrics in Grafana
 - Six alert rules configured and tested
 - `docs/OPERATIONS.md` updated with runbook sections
 - Verified end-to-end: trigger a failure → Sentry captures it → Grafana alert fires → runbook explains what to do
-

Spec Part 2 · 2026-06-18 · Claude.ai Build after: WebSocket spec (Part 1) stable on AWS

References: `docs/AWS_MIGRATION_PLAN.md` Phase 5 (Hardening)